

SkyBridge Compass: A Crypto-Agile P2P Handshake Framework for Remote Desktop and File Transfer with Auditable Downgrade Policy and Post-Quantum Readiness

Zi'ang Li, Peng Liu

Abstract—Peer-to-peer remote desktop and file transfer systems increasingly operate across heterogeneous devices, untrusted networks, and long lifecycles in which cryptographic assumptions may expire faster than products. Yet “post-quantum readiness” is often treated as a one-off algorithm swap rather than a migration problem with downgrade resistance, auditability, and reproducibility requirements. This paper presents a migration-focused framework, instantiated in SkyBridge Compass, that explicitly separates (i) negotiated cryptographic suites, (ii) provider backends, and (iii) a policy layer that governs when (and whether) classic fallback is permissible. We use an actor-isolated handshake state machine with deterministic outcomes and structured security-event emission, enabling end-to-end audit trails for downgrade, identity mismatch, and signature verification failures. We evaluate three configurations on macOS 26.x as a deployment instance: classic (X25519 + Ed25519), PQC via liboqs (ML-KEM-768 + ML-DSA-65), and PQC via native CryptoKit (ML-KEM-768 + ML-DSA-65). We do not claim perfect forward secrecy for the KEM-based PQC suites in v1; post-quantum confidentiality holds only while long-term KEM private keys remain uncompromised. End-to-end handshake completion latency (including event emission overhead) is 2.454 ms (p95 2.586 ms) for classic, 2.976 ms (p95 3.656 ms) for liboqs PQC, and 5.443 ms (p95 6.272 ms) for CryptoKit PQC under the benchmark controls (Classic/liboqs: N=1000; CryptoKit: N=5000). The handshake wire size (MessageA + MessageB + 2×Finished) grows from 687 B (classic) to 12002 B (PQC). Beyond performance, we validate downgrade policies, failure semantics, and audit-signal fidelity under fault injection, and we release scripts and artifacts that reproduce all primary figures and tables. Our portability claim is mechanism-level (protocol/policy contract and verification workflow); reported latency/overhead values are deployment-instance measurements.

Index Terms—post-quantum cryptography, crypto agility, downgrade resistance, secure handshake, peer-to-peer systems, remote desktop, auditability, reproducibility, platform deployment.



1 INTRODUCTION

Peer-to-peer (P2P) remote desktop and file transfer systems must establish secure sessions under adversarial network conditions, device churn, and multi-year lifecycles. These systems face a practical tension: users demand “it just works” pairing across heterogeneous stacks, while defenders require that cryptographic negotiation remains robust against downgrade and implementation-level failure modes. Post-quantum cryptography (PQC) increases this tension. Larger messages, higher computational cost, and uneven platform support make fallback tempting—yet fallback is exactly where silent downgrade and unverifiable behavior tend to hide.

Existing secure-channel designs and frameworks provide mature cryptographic building blocks, but they do not, by default, solve the migration problem an implementer actually faces: (i) selecting among classic/PQC/hybrid suites across platform strata, (ii) ensuring that policy (e.g., “require PQC”) is enforced at the entry points where callers cannot accidentally violate it, (iii) targeting deterministic failure semantics under timeouts, reorder/duplication, and malformed inputs, and (iv) emitting security-relevant telemetry that can be audited without turning the system into a logging minefield. In practice, many systems either hard-code a single suite, or allow “compatibility” to creep

into negotiation in ways that are hard to reason about and harder to verify.

This paper addresses these gaps with a migration-first framework, instantiated in SkyBridge Compass, for explicit PQC migration and auditability, with Apple platforms as a deployment instance. The core idea is to treat crypto agility as a layered contract: negotiated suites define the protocol surface; provider backends implement suites using platform-native or portable cryptography; and a policy layer governs fallback and produces structured evidence when (and only when) downgrade occurs. We complement this design with an actor-isolated handshake driver that enforces one-shot completion, timeouts, and sensitive-material zeroization, yielding deterministic outcomes under the modeled transport faults. Operationally, the strict policy now has two explicit realizations: *Compliance Mode* (no classic path at all) and *Bootstrap-Assisted Mode* (one-time classic control channel only to refresh paired KEM identity keys, followed by mandatory PQC rekey before business traffic is released). We separate mechanism-level claims (policy-gated downgrade behavior, deterministic failure semantics, and auditability) from deployment-level claims (absolute latency/throughput), and evaluate each layer with corresponding evidence.

Verifiable Security Guarantees. To reduce ambiguity in reviewer-facing claims, we state the v1 guarantees as a small set of *verifiable* properties under the threat model in Section 5;

- Zi'ang Li is with Independent Researcher, Tianjin, China. E-mail: 2403871950@qq.com.
- Peng Liu is with Independent Researcher, Qiqihar, China.

the Supplementary Materials provide additional guarantees and artifact-level evidence pointers.

- **Negotiation integrity:** the Responder selects only a suite offered by the Initiator and with a matching key share; unknown suite IDs are rejected if they appear in `keyShares[]` or as `selectedSuite`. *Evidence:* Fig. 1; Table 3.
- **Transcript binding:** `sigB` commits to `MessageA` and the chosen suite (and policy), so any tampering of `supportedSuites[]`, `keyShares[]`, or `policy` fails verification. *Evidence:* Table 3.
- **Downgrade resistance and auditability (policy-defined):** strict policy is explicit and auditable in two forms: *Compliance Mode* forbids classic establishment entirely; *Bootstrap-Assisted Mode* permits a one-time classic control channel only for paired KEM-key recovery and requires PQC rekey before business traffic. The default policy permits classic fallback only for a local-only whitelist of capability/provider failures (timeouts and unauthenticated network-derived failures cannot trigger default-mode fallback). Every fallback emits a structured `cryptoDowngrade` event. *Evidence:* Fig. 4; Fig. 5; Table 7; Fig. 7.

We additionally provide explicit key confirmation via two Finished frames (Fig. 1) and document v1 non-claims and limitations in Section 8.

1.1 Contributions

Contributions:

- 1) **C1: Formalized migration security contract.** We formalize a migration security contract and define four experiment families (authentication, negotiation, downgrade, hybrid forward secrecy). We state theorem/proposition guarantees under standard assumptions (KEM IND-CCA, SIG EUF-CMA, HKDF PRF) and provide a machine-checkable Tamarin artifact aligned with implementation-level claims. *Evidence:* Section 5; Table 3; `formal/skybridge_v2.spthy`.
- 2) **C2: Protocol v2 upgrade with backward-compatible bridge.** We introduce a forward-secrecy-oriented v2 suite that composes a static KEM secret with an additional ephemeral contribution while retaining v1 interop through policy-controlled bridge negotiation. This provides stronger post-compromise secrecy semantics than v1 but is not claimed as full ephemeral-ephemeral PFS. *Evidence:* Fig. 1; `HandshakeV2PFSTests`; `ProtocolSignatureRegressionTests`.
- 3) **C3: Reproducible system realization with auditable telemetry.** We ship an actor-isolated implementation with tamper-evident audit telemetry anchored to transcript hashes. We also release artifact pipelines reporting distributional metrics (p50/p95/p99, CI/effect-size) rather than point estimates only. *Evidence:* Fig. 4; Table 9; `AuditTrailTests`; `HandshakeBenchmarkTests`.

1.2 Paper Organization

Section 2 reviews related work on P2P pairing, negotiation, and PQC migration. Section 3 details system architecture and protocol design. Section 4 presents the handshake state machine. Section 5 presents the security model and guarantees. Section 6 describes implementation highlights and platform instantiation. Section 7 outlines evaluation methodology and metrics. Section 8

discusses limitations and future work, and Section 9 concludes. Supplementary Materials provide full wire-format definitions, implementation details, and extended tables.

2 RELATED WORK

We use the following terms throughout: *crypto agility* (ability to migrate primitives without redesign), *downgrade resistance* (prevent or detect forced fallback), *suite negotiation* (explicit, authenticated selection of handshake KEM/SIG/AEAD/KDF), *PQC readiness* (ability to deploy PQC alongside classic suites), and *auditability* (deterministic, minimal event traces for security decisions). Our threat model assumes an active network attacker attempting to induce fallback or obscure failures; therefore we focus on mechanisms that make negotiation and failures explicit and auditable.

2.1 Peer-to-Peer Device Pairing

Pairing protocols such as Bluetooth SSP [1], Continuity [2], and PAKEs like SPAKE2+ [3] establish initial trust but stop short of post-pairing migration, downgrade policy, or auditability. SkyBridge Compass begins *after* pairing and treats pinned identity keys as inputs rather than outcomes.

2.2 Cryptographic Agility and Negotiation

TLS 1.3 and QUIC bind negotiated parameters into transcripts and transport parameters to prevent silent downgrade [4], [5], but they do not define an auditable, system-level fallback policy across heterogeneous providers. We operationalize migration guidance [6], [7] with explicit policy gates and event emission that make downgrade decisions observable and testable.

2.3 Handshake Frameworks and Noise-Style Patterns

Noise provides a compact framework for describing authenticated key exchange patterns with explicit transcript binding and identity key usage [8]. While SkyBridge Compass does not implement Noise directly, our `MessageA/MessageB/Finished` exchange targets similar transcript-binding goals while adding explicit suite negotiation and auditable policy outcomes.

2.4 Post-Quantum Cryptography Deployment

PQC standardization and early deployments highlight size and performance costs, along with pressure for hybrid deployments [9], [10], [11], [28], [29], [12], [30], [31], [33], [13], [14], [15]. We focus on making downgrade policy and auditability explicit across providers, treating Apple’s CryptoKit PQC [16], [23] as a deployment instance rather than a protocol dependency.

2.5 Secure State Machine Design

State machine vulnerabilities have been a persistent source of security bugs in protocol implementations [17], [18]. Actor-based concurrency models, as implemented in Swift’s actor system [19], provide compile-time guarantees against data races. We extend this model with deterministic completion, timeout handling, and audit events for cryptographic handshakes.

TABLE 1

Baseline comparison with widely deployed handshakes. Message counts and negotiation semantics are summarized from the TLS/QUIC/DTLS RFCs and Noise specification [4], [5], [8], [20]. Loopback latencies and wire sizes are measured with BaselineBenchRunner (release), N=1000; wire sizes report loopback p50/p95 (kB) and are cert-dependent for TLS/QUIC/DTLS. Noise wire size is pattern-dependent and reflects the XX pattern used in this baseline.

Protocol	Suite nego. explicit	Auditable downgrade policy	Failure semantics	Msgs	Wire size (p50/p95, kB)	p50/p95 latency
TLS 1.3 [4]	Yes (cipher suites, groups)	Downgrade prevention only; audit external	Alerts defined; audit external	4 (1 RTT)	8.6/8.7	8.53/14.01 ms
QUIC [5]	Yes (TLS 1.3)	Same as TLS 1.3	Same as TLS 1.3	4 (1 RTT)	7.6/14.1	0.34/11.37 ms
WebRTC DTLS [20]	Yes (DTLS 1.2 ciphers)	Downgrade prevention only; audit external	Alerts defined; audit external	4 (1 RTT)	3.0/3.1	8.18/11.79 ms
Noise XX [8]	No (fixed pattern)	No policy layer	Pattern-defined; audit external	3	2.5/2.6	0.44/0.58 ms
					4.8/4.9 (classic)	1.44/1.57 ms (classic)
					36.9/43.6 (liboqs)	1.95/2.76 ms (liboqs)
SkyBridge Compass	Yes (suite + policy bound)	Yes (explicit policy + events)	Deterministic + evented	4	18.6/18.8 (CryptoKit)	5.91/7.53 ms (CryptoKit)

2.6 Baseline Comparison with Widely Deployed Handshakes

Table 1 contrasts SkyBridge Compass with widely deployed handshake frameworks on explicit negotiation, downgrade auditability, and failure semantics, and reports message count, wire size, and latency where available.

BaselineBenchRunner measures *time-to-ready* (NWConnection.ready) for fresh loopback connections; each sample creates a new connection and records the elapsed time from connect start to ready. We report N=1000 after 10 warmup iterations (warm cache); 0-RTT/session resumption is not explicitly enabled and results reflect Network.framework defaults. QUIC measurements use ALPN sbq with a single bidirectional stream, and no application data is sent (BASELINE_KICKOFF_BYTES=0).

Loopback SkyBridge p95 latency is 1.57/2.76/7.53 ms for classic/liboqs/CryptoKit in our loopback harness; these numbers are not directly comparable to TLS/QUIC/DTLS handshakes in production deployments, but provide a reproducible baseline under a consistent measurement setup. The in-memory handshake microbench isolates crypto/state-machine overhead (p95 2.18/4.18/6.16 ms for classic/liboqs/CryptoKit) and is reported in Section 7 and Supplementary Tables S1–S2. Loopback TLS/QUIC/DTLS p95 latency is 11.37–14.01 ms; wire-size baselines (p50/p95) are reported in Supplementary Table S11 and are reproducible via loopback pcap capture (Scripts/run_baseline_bench.sh). SkyBridge-CryptoKit loopback baselines are included in Supplementary Table S11.

3 SYSTEM ARCHITECTURE

3.1 Overview

SkyBridge Compass implements a layered architecture separating concerns across four primary domains: discovery, cryptographic operations, handshake management, and session transport. Fig. 2 illustrates the high-level component relationships and trust boundaries.

3.2 Threat Model

We model an active network attacker in the Dolev-Yao style with the following **capability set**:

Network Capabilities (Attacker-Controlled):

- **Drop / delay / reorder / duplicate** packets on the discovery channel
- **Modify/inject** arbitrary bytes in MessageA, MessageB, or Finished frames
- **Replay** prior handshake messages across sessions
- **Spoof capability or policy claims** to induce weaker negotiation
- **Force negotiation failure** by corrupting suites, key shares, or signatures

Defense Mapping (Protocol + Policy):

- **Transcript binding:** sigB covers MessageA and the chosen suite, so modifications to supportedSuites[], keyShares[], or policy fail verification.
- **Negotiation integrity:** Responder MUST select a suite that Initiator offered and for which a key share exists; otherwise reject missingKeyShare.
- **Replay control:** handshakeId is derived from nonces and cached to reject duplicates within a window.
- **Downgrade resistance:** timeout-triggered *default-mode* fallback is disallowed; only whitelisted local capability errors may fallback under default policy. Strict policy offers either Compliance Mode (no classic establishment) or Bootstrap-Assisted Mode (classic control-only bootstrap with mandatory PQC rekey before business data).
- **Rate limiting:** per-peer fallback cooldown prevents rapid downgrade cycling.
- **Legacy gating:** legacy P-256 acceptance requires an authenticated channel or an existing trust record.

Out-of-Scope (Baseline Claims):

- **Traffic analysis** from size/timing metadata is not assumed in the baseline threat model. SBP2 is an optional mitigation evaluated separately in Section 7, and its overhead/privacy tradeoffs are reported without changing the core security claims.

Trust Assumptions:

- 1) The initial pairing ceremony occurs over a trusted out-of-band channel (e.g., visual PIN comparison on both device screens)
- 2) The device Keychain provides integrity guarantees for stored identity keys

- 3) The Secure Enclave (when available) provides hardware-backed key isolation that resists software-level extraction
- 4) Users can visually verify device identity during pairing (no blind trust)

Out of Scope:

- Side-channel attacks on cryptographic implementations
- Physical attacks on Secure Enclave hardware
- Compromise of the operating system kernel
- Social engineering attacks on users

3.3 Protocol Message Flow

Fig. 1 illustrates the handshake message sequence and transcript coverage.

MessageA carries the `supportedSuites[]` (offered suites) and `keyShares[]` arrays (up to two entries to bound size); MessageB selects a suite and binds MessageA via `sigB`. Both sides verify that the selected suite was offered and that a matching key share exists, preventing negotiation mismatch. Under the two-attempt strategy (Section 6.7), PQC and classic are not co-offered in a single MessageA; when fallback is permitted, the Initiator retries with a classic-only MessageA after a whitelist of locally generated capability and negotiation failures. Under strict policy, an optional Bootstrap-Assisted recovery path may transiently establish a classic-only control channel to refresh paired KEM identity keys, but business traffic remains gated until a successful PQC rekey. Explicit key confirmation uses two small Finished frames to avoid half-open state. Full wire-format definitions, field validation rules, and encoding rationale appear in the Supplementary Materials (Wire Format and Validation Rules).

Failure Semantics:

- All failures trigger `HandshakeContext.zeroize()` before error propagation
- Security events are emitted for audit logging
- No partial state is retained after failure

Implementation alignment (artifact snapshot, 2026-01-23).

In the current code revision, connection orchestration for the

remote desktop/file-transfer path is route-aware and authentication-gated: local discovery can prefer USB-capable host endpoints, control-channel connections are promoted to “connected” only after handshake authentication, and active file-transfer routing prioritizes live authenticated sessions before fallback candidates. These implementation surfaces are enumerated in the Supplementary Artifact Map.

3.4 CryptoProvider Protocol

The CryptoProvider protocol defines a unified interface for all cryptographic operations required by the handshake and session layers [21]. This abstraction enables transparent substitution of underlying implementations without modifying caller code.

The `SigningKeyHandle` abstraction resolves the apparent conflict between “private key as Data” and “Secure Enclave keys never leave hardware.” Software keys are one variant; hardware-backed keys use a reference or callback, ensuring the protocol interface does not assume exportability.

The protocol mandates `Sendable` conformance, ensuring thread-safe usage across Swift’s structured concurrency model. Each provider exposes its tier classification (`nativePQC`, `liboqsPQC`, or `classic`) and active cipher suite, enabling runtime introspection for logging and policy enforcement.

3.5 CryptoSuite Wire Format

A **suite** defines the handshake tuple (KEM, SIG, AEAD, KDF) used for the KEM-DEM envelope and transcript binding. Data-plane AEAD is negotiated separately and is fixed to AES-256-GCM in v1 (Table 11). Suite identifiers use a compact 16-bit wire ID with reserved ranges for hybrid, PQC, classic, and experimental suites to enable forward compatibility. The full range table, encoding rules, and X-Wing notes appear in the Supplementary Materials (Suite Identifiers and Wire Format). Unknown suite identifiers are parsed as `unknown(wireId)` for audit visibility and forward compatibility; negotiation rejects unknown values if they appear in `keyShares` or as `selectedSuite`.

3.6 Provider Factory and Selection

The `CryptoProviderFactory` implements deterministic provider selection based on runtime capability detection and policy configuration:

The selection algorithm proceeds as follows:

- 1) **Capability Detection:** Query the environment for native PQC availability and liboqs presence.
- 2) **Policy Application:** Match detected capabilities against the requested policy.
- 3) **Provider Instantiation:** Create the appropriate provider instance for the detected environment and policy.
- 4) **Event Emission:** Emit a `SecurityEvent` recording the selection decision.

For `preferPQC` policy, the precedence order is `ApplePQCProvider` → `OQSPQCProvider` → `ClassicProvider`. This order is deterministic. The `requirePQC` policy returns an `UnavailablePQCProvider`. It throws on all operations if no PQC implementation is available.

Terminology alignment. We use two orthogonal knobs: provider selection (`preferPQC/requirePQC`) and handshake

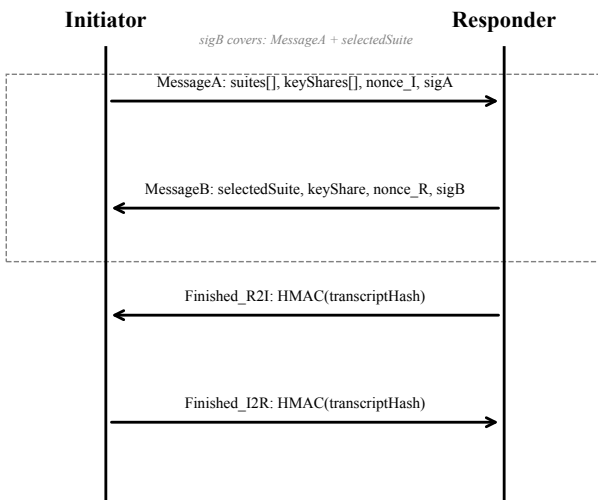


Fig. 1. Handshake sequence with transcript coverage and policy visibility.

TABLE 2
Suite negotiation scenarios under the two-attempt strategy (homogeneous per attempt).

Initiator attempt(s)	Responder Capability	Outcome	Notes
Attempt 1: PQC-only	PQC available	PQC established	Best available selected
Attempt 1: PQC-only; Attempt 2: classic-only	Classic only (no PQC material)	Classic established	Graceful fallback (evented)
PQC-only (<i>strictPQC</i> , bootstrap-assisted)	Missing/stale paired KEM key	PQC established after rekey	Classic channel is control-only before rekey
PQC-only (<i>strictPQC</i>)	Classic only	Handshake failed	Policy enforced (no fallback)
Classic-only	PQC available	Classic established	Initiator policy respected

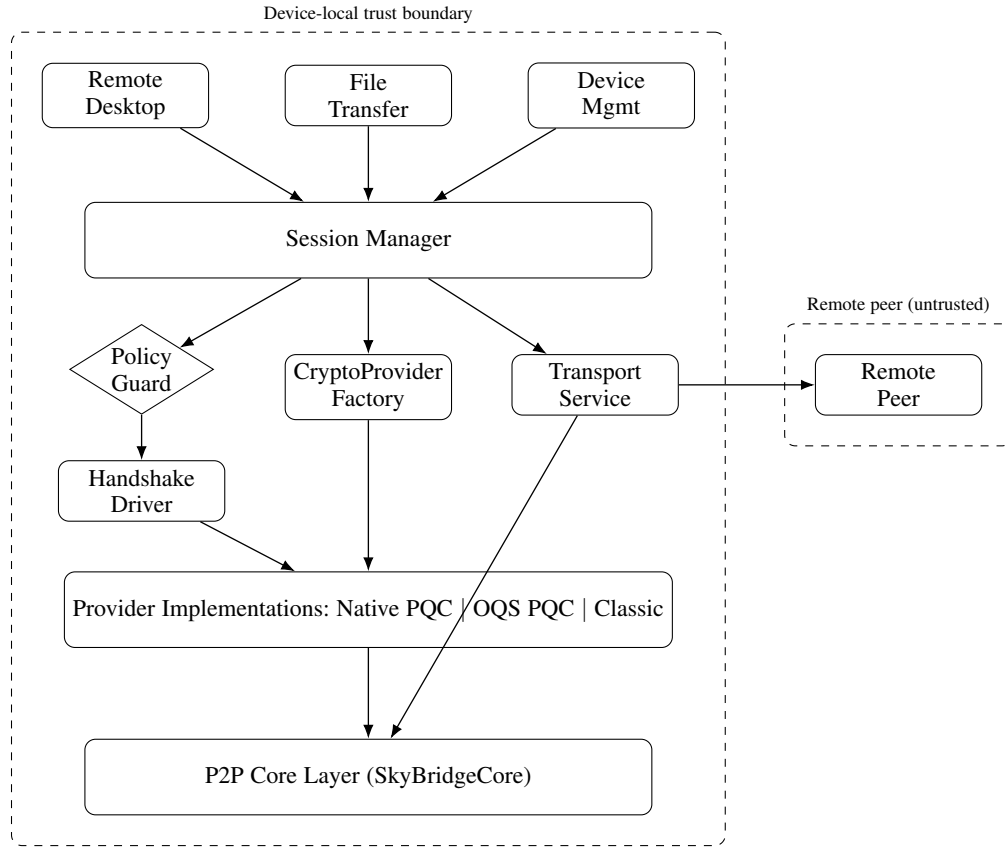


Fig. 2. SkyBridge Compass system architecture showing device-local trust boundary (dashed) and policy guard (diamond).

fallback policy (default/*strictPQC*). In our evaluation, default corresponds to prefer-PQC with evented classic fallback permitted, while *strictPQC* requires PQC with either (i) Compliance Mode (no classic establishment) or (ii) Bootstrap-Assisted Mode (control-only bootstrap plus mandatory PQC rekey).

3.7 KEM-DEM Authenticated Encryption Format

We use a KEM-DEM envelope (KEM $\rightarrow$ HKDF $\rightarrow$ AEAD) as a compatibility layer when native HPKE is unavailable, binding role and transcript into the key schedule with explicit length limits for DoS protection. On Apple 26+ platforms, we use native CryptoKit PQC primitives (ML-KEM/ML-DSA) within the same envelope to preserve a single wire format across platforms. Full

header/body formats, versioning, and length limits are provided in the Supplementary Materials (KEM-DEM Envelope).

Why not direct RFC 9180 HPKE everywhere (v1)? Three engineering constraints motivate this compatibility layer in v1: (i) a unified transcript/signature binding surface across mixed provider stacks, (ii) strict backward compatibility with deployed v1 bridge behavior, and (iii) deterministic parser-level DoS checks before provider dispatch. When native HPKE is available (Apple 26+), our v2 path uses RFC 9180 HPKE directly; thus the v1 KEM-DEM envelope is transition code, not a claim of new cryptographic primitive design.

4 HANDSHAKE STATE MACHINE

4.1 Design Principles

The handshake subsystem addresses three critical requirements:

- 1) **Race Condition Prevention:** Concurrent message arrival and timeout expiration must not cause double-resume of continuations.
- 2) **Sensitive Material Protection:** Ephemeral private keys and shared secrets must be zeroized on all exit paths.
- 3) **Observability:** All state transitions and failures must emit structured events.

4.2 Actor-Isolated Architecture

The `HandshakeDriver` actor manages handshake state with compile-time data race prevention:

The `HandshakeContext` actor isolates sensitive cryptographic material:

Reentrancy Considerations. Swift actors permit method re-entry at suspension points (`await`). While actor isolation prevents data races, it does not prevent interleaving of method executions. Our `HandshakeDriver` mitigates reentrancy risks by routing completion through `finishOnce(with:)`, buffering early results, canceling timeouts on completion, and advancing state before `await` points.

Await-window limitation. The `handleMessageB` method awaits `cryptoProvider` operations inside the actor. If a new message arrives during this await, the actor could theoretically process it before the current operation completes. We mitigate this by advancing state to `.processingMessageB` before the await, copying immutable inputs, validating a monotonic epoch counter after resumption, and emitting security events on unexpected transitions.

This approach provides structural protection against reentrancy hazards rather than relying on timing assumptions. A formal analysis using model checking is planned for future work.

4.3 State Transitions

Fig. 3 illustrates the handshake state machine for both roles.

4.4 Double-Resume Prevention

The `finishOnce` method provides a single convergence point for handshake completion, handling the race between timeout expiration and message arrival by: (1) canceling the timeout task on any completion, (2) guarding continuation access with immediate nil assignment, and (3) buffering early results for late continuation establishment.

4.5 Secure Zeroization

Swift's standard `Data` and `Array` types use copy-on-write (COW) semantics, which can leave uncontrolled copies of sensitive material in memory. The `SecureBytes` class addresses this by using manual `UnsafeMutableRawPointer` allocation to maintain exclusive ownership, preventing COW-induced secret proliferation. All failure paths explicitly call `context.zeroize()` before error propagation, with `deinit` zeroization as defense-in-depth. On Darwin platforms, zeroization uses platform-supported secure wipe primitives (e.g., `explicit_bzero` or `memset_s`) where available. We avoid exporting secrets as `Data` when possible, but acknowledge that

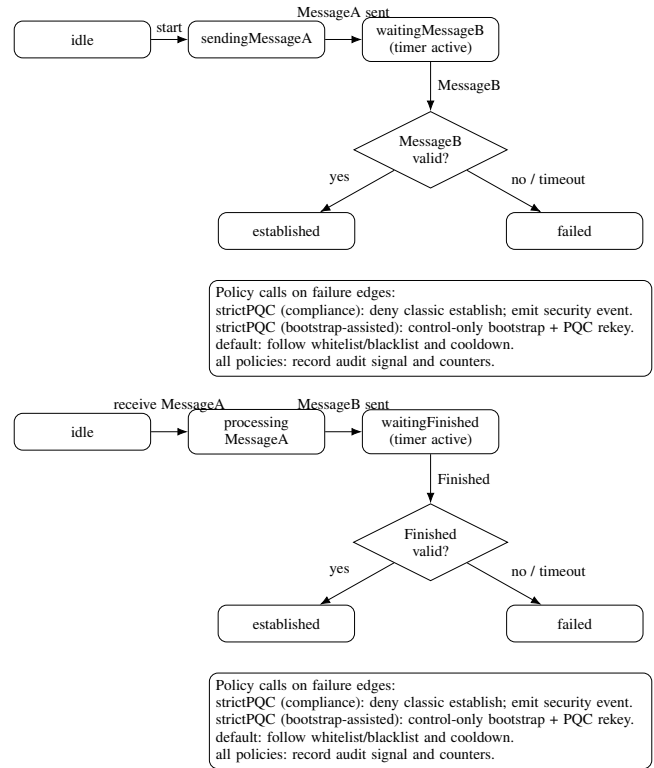


Fig. 3. Handshake state machines for Initiator (top) and Responder (bottom).

OS-level snapshots, API boundary copies, or privileged inspection can still retain residues; zeroization is a defense-in-depth measure rather than a sole security boundary.

4.6 Transcript Binding and Signature Separation

The protocol uses domain-separated signatures to prevent cross-message and cross-session replay attacks.

Signature A (MessageA) is computed by the Initiator over the `MessageA`-local fields, binding suite preference, policy intent, and identity context before any responder data is received:

For Signature A, the initiator signs `SkyBridge-A`, protocol version, canonical suite/key-share set, client nonce, capabilities, policy, and initiator identity key. Optional Secure Enclave binding uses `SkyBridge-SE-A`.

Signature B (MessageB): The Responder signs a transcript that commits to `MessageA`:

For Signature B, the responder signs `SkyBridge-B`, `transcriptA`, selected suite wire ID, responder key share, server nonce, payload hash, and responder identity key. Optional Secure Enclave binding uses `SkyBridge-SE-B`.

Key Derivation: Nonces, transcripts, suite binding, and *directional separation* are included in the KDF, and the salt is transcript-bound (non-empty):

KDF input concatenates the `SkyBridge-KDF` label, suite wire ID, transcript hashes, and nonces. The salt uses the domain string `SkyBridge-KDF-Salt-v1|`. Direction labels separate traffic classes:

- `initiator_to_responder`
- `responder_to_initiator`

This structure is designed to ensure: (1) `sigB` commits to `MessageA`, so any tampering breaks verification; (2) domain

separators prevent cross-protocol attacks; (3) SE signatures are session-bound and non-replayable.

5 SECURITY MODEL AND GUARANTEES

5.1 Security Model

We assume an active network adversary with full control of the discovery channel (drop, delay, reorder, duplicate, modify, and inject messages) but without breaking standard cryptographic assumptions. The attacker can replay prior handshakes and attempt downgrade by suppressing PQC-related fields. We assume the out-of-band pairing ceremony is trusted and that device key storage (Keychain/Secure Enclave) preserves key integrity. The attacker may compromise a peer after pairing and learn long-term signing key material, but cannot retroactively forge signatures for past sessions. We do not claim forward secrecy for KEM-based suites in v1: ML-KEM encapsulation targets a peer’s long-term KEM public key obtained during pairing. Accordingly, post-quantum confidentiality against store-now-decrypt-later adversaries holds only while long-term KEM private keys remain uncompromised; later compromise of the corresponding KEM private key can enable decryption of recorded sessions.

Assumptions:

- The pairing ceremony delivers the correct peer identity key (PIN or visual verification is not bypassed).
- Pinned identity keys stored in Keychain/Secure Enclave retain integrity across normal device operation.
- Devices remain uncompromised (no root or malware control) during handshake execution.

If assumptions fail:

- If pairing is subverted and the pinned identity key is replaced, peer authentication is no longer meaningful; the system must require re-pairing and treat the session as untrusted.
- If device integrity is lost, confidentiality and key secrecy cannot be guaranteed; the system should abort and emit high-severity events such as `identityMismatch` or `handshakeFailed`.

5.2 Security Goals, Formal Definitions, and Proof Sketches

Definitions.

- **D1 (Forced downgrade / forced fallback).** A downgrade occurs if a business-capable session is established with a classic suite when strict policy requires PQC, or if classic fallback is accepted under the default policy without satisfying the local-only fallback gate (Definition D2) and without emitting the corresponding audit event (Property G8).
- **D2 (Local-only fallback gate, default policy).** Fallback is permitted only after (i) the peer identity is verified against the pinned key (paired peers) and (ii) the triggering failure is locally generated capability/instantiation unavailability (whitelist), excluding timeout and any unauthenticated network-derived failure.
- **D3 (Audit evidence).** The minimal sufficient evidence set is the tuple {event type, timestamp, deviceId, suiteId, failure reason code, policy mode, fallback flag, cooldown remaining} (Property G8).

- **D4 (Bootstrap-Assisted strict recovery gate).** In strict policy, a temporary classic control channel is allowed only for paired KEM-key recovery triggers (`missingKEM`, `staleKEMRecovery`); non-control business messages are blocked until PQC rekey succeeds.

We state the core properties as theorem/proposition-level claims tied to standard assumptions and protocol mechanics. To reduce vacuity risk, the released Tamarin artifact models explicit classic/PQC/v2 traces, explicit fallback-gate and downgrade events, and includes existence lemmas in addition to safety lemmas (`formal/skybridge_v2.spthy`) [32].

Formal security definitions (experiment style).

- Exp_{auth} : adversary wins if it causes a paired endpoint to accept an unauthenticated peer identity.
- Exp_{neg} : adversary wins if `selectedSuite` is accepted outside the signed/advertised offer context.
- Exp_{dg} : adversary wins if classic establishment occurs in violation of strict policy, or under default policy without local fallback gate + downgrade event.
- $\text{Exp}_{\text{fs-hybrid}}$: for v2, adversary wins if it computes session keys without both static KEM contribution and ephemeral contribution under transcript binding.

Assumption set.

- KEM IND-CCA security for negotiated KEM suite (ML-KEM/X-Wing profile).
- Signature EUF-CMA security for protocol signatures (ML-DSA-65 / Ed25519 as configured).
- HKDF-SHA256 as a PRF in the used extraction/expansion domain.

Theorem T1 (Authentication). Under EUF-CMA and trusted pairing pins, advantage in Exp_{auth} is negligible up to bounded implementation error terms (e.g., policy misconfiguration outside the model).

Theorem T2 (Negotiation integrity). Under EUF-CMA and transcript binding, advantage in Exp_{neg} is negligible.

Proposition P1 (Policy-gated downgrade). Under the threat model in Section 5, timeout/drop/reorder adversaries cannot force successful classic establishment in strict policy, and cannot produce silent classic fallback in default policy without the local gate event.

Proposition P2 (v2 hybrid FS semantics). For v2 sessions, compromise of long-term KEM private key alone is insufficient to reconstruct the derived session root unless the ephemeral contribution secret is also recovered. This is an intermediate secrecy guarantee and is intentionally weaker than full ephemeral-ephemeral PFS.

Property G1 (Negotiation Integrity): The selected suite must be among the Initiator’s offered suites and have a corresponding key share.

Proof sketch:

- `MessageB` includes `selectedSuite`, and `sigB` covers `MessageA`.
- Any modification to the offered suite list or key-share list breaks `sigB` verification.
- The Responder validates `selectedSuite` against the offered suites and checks that a matching key share is present.

Property G2 (Mutual Authentication for Paired Peers): For previously paired devices, the peer identity is authenticated and bound to the handshake transcript.

Proof sketch:

- `identityPubKey` is pinned during OOB pairing.
- Signatures in `MessageA` and `MessageB` over transcript data must verify under the pinned key; otherwise `identityMismatch` aborts the handshake.

Property G3 (Session Key Secrecy): The derived session keys remain secret under standard KEM/DH and signature security assumptions. For KEM-based PQC suites in v1, we do not claim post-compromise forward secrecy: confidentiality holds only while long-term KEM private keys remain uncompromised. In key-control terms, v1 KEM establishment is initiator-contributory to a responder static key; v2 adds explicit responder ephemeral contribution to reduce unilateral key-control concentration.

Proof sketch:

- The shared secret is derived from KEM decapsulation or DH, and keys are extracted via HKDF with transcript-bound salt.
- An attacker without the private key material cannot compute the shared secret or derive session keys.

Property G4 (Replay Resistance): Replayed handshakes do not establish a session.

Proof sketch: `handshakeId` derives from fresh nonces and suite identifiers; replay detection caches recent IDs and rejects duplicates within the window.

Property G5 (Strict-Policy Consistency): Strict policy never permits a business-capable classic session. Compliance Mode has no classic path. Bootstrap-Assisted Mode may use a transient classic control channel, but business traffic is blocked until PQC rekey completes.

Proof sketch:

- `TwoAttemptHandshakeManager` enforces a policy gate before any default-mode fallback attempt.
- Strict compliance removes classic establishment edges.
- Strict recovery trigger-limits bootstrap, emits explicit `handshake_fallback` events, and keeps messaging control-only until PQC rekey succeeds.

We validate this behavior using policy downgrade benchmarks (Fig. 4) and bootstrap control-gating tests.

Property G6 (Safe Fallback under default policy): Fallback can only occur after peer identity is verified under the pinned key and only for a whitelisted set of locally generated capability errors; timeout and network-derived failures never trigger fallback.

Proof sketch: Fallback is gated after identity verification; the whitelist is evaluated on locally generated capability/instantiation errors rather than unauthenticated network input. `timeout` is explicitly blocked and per-peer cooldown limits repeated downgrades. *Proposition (No network-induced suiteNegotiation fallback):* Under Definition D2, `suiteNegotiationFailed` can trigger default-mode fallback only in the local pre-send branch (no local keyshare constructible for the offered PQC suite). If the failure is observed after remote input, it is treated as network-derived and cannot satisfy the fallback gate. *Proposition (No forced fallback by packet loss):* Under the attacker model in Section 5, an active network attacker who can drop/delay/reorder

messages cannot induce a successful classic session by causing timeouts, because timeout-triggered fallback is disallowed and the default-policy fallback gate (Definition D2) requires locally generated unavailability after identity verification.

Property G7 (Legacy Acceptance Preconditions): Legacy P-256 signatures are accepted only under authenticated pairing or an existing trust record.

Proof sketch: Legacy acceptance is gated by `LegacyTrustPrecondition`; pure network stranger connections fail. This is validated by the precondition test matrix (Table 7).

Property G8 (Auditability): Cryptographic downgrades and exceptional states are observable via a minimal sufficient information set.

Proof sketch: The handshake emits `cryptoDowngrade` and related events with reason, `deviceId`, and cooldown context, and records them into a per-session hash-chain (`AuditTrail`) anchored to the handshake transcript hash, enabling post-hoc verification of policy adherence and event-order integrity.

Audit field requirements:

- **MUST log** (minimal sufficient set): event type, timestamp, `deviceId`, `suiteId`, failure reason code, policy mode, fallback triggered (bool), cooldown remaining.
- **MUST NOT log** (privacy preservation): session keys, nonces, raw signatures, message payloads, user identifiers beyond `deviceId`.
- **Tamper-evidence:** In addition to synchronous event emission, each session maintains a transcript-anchored audit hash-chain (`AuditTrail`). This provides cryptographic tamper evidence for in-session event order/content under non-compromised runtime assumptions. System log retention (`os_log`) remains best-effort and can still be truncated by privileged adversaries.

5.3 Complexity and Overhead Bounds

Let S and K denote implementation bounds for offered suites and key shares (`maxSupportedSuites`, `maxKeyShareCount`); currently $S \leq 8$ and $K \leq 2$. Encoding/decoding work for `MessageA/MessageB` is:

$$T_{\text{codec}} = O(S + K + |M_A| + |M_B|),$$

with constant-time checks on fixed-length fields and linear passes over suite/key-share vectors.

Handshake cryptographic operation counts are:

- **v1 PQC path:** $1 \times$ KEM encapsulate + $1 \times$ KEM decapsulate + 2 signatures + 2 signature verifies + HKDF key schedule.
- **v2 FS path:** v1 counts plus one ephemeral contribution generation per side and one additional shared-secret composition step (HKDF extract-expand on static+ephemeral input).
- **Classic path:** one DH-style encapsulation/open path (KEM-DEM abstraction), signatures/verifications, and HKDF key schedule.

Closed-form byte overhead between v1 and v2 is dominated by the extra contribution fields. Let c_I denote the initiator contribution carried in `MessageA` and $c_R^{(v1)}, c_R^{(v2)}$ denote responder-share fields in `MessageB` for v1 and v2:

$$\Delta|M_A| = 2 + |c_I|, \quad \Delta|M_B| = |c_R^{(v2)}| - |c_R^{(v1)}|.$$

In current implementation (X25519 contribution, 32 bytes): $\Delta|M_A| = 34$ bytes and $\Delta|M_B| = 32$ bytes.

6 IMPLEMENTATION

Implementation references (source files, line numbers) are consolidated in the Supplementary Materials (Artifact Map).

6.1 Platform Support Matrix

SkyBridge Compass uses a tiered provider model: a native PQC provider when available (e.g., CryptoKit ML-KEM/ML-DSA), a portable liboqs provider, and a classic provider. Provider selection is deterministic and prefers the strongest available tier; Secure Enclave proof-of-possession is optional and orthogonal to the negotiated suite. The full Apple provider matrix and the cross-platform runtime suite-policy matrix are in the Supplementary Materials (Tables S18 and S20).

6.2 Native PQC Integration

The native provider wraps CryptoKit ML-KEM/ML-DSA APIs [16], [23] when the SDK exposes them, enabling platform-backed PQC suites and HPKE-based encapsulation. We treat this as a deployment instance rather than a protocol dependency; integration details are provided in the Supplementary Materials (Native PQC Integration).

6.3 Conditional Compilation Strategy

PQC symbols are gated behind build-time flags to keep the codebase buildable on SDKs that lack PQC types and to avoid compile-time failures. The flagging strategy and build settings are detailed in the Supplementary Materials (Conditional Compilation).

6.4 Security Event Emission

All cryptographic decisions emit structured audit events with backpressure and rate limiting; implementation details are in the Supplementary Materials (Security Event Emission).

6.5 Secure Enclave Integration

For hardware-backed signing, a `SigningCallback` protocol allows Secure Enclave keys without exposing private material. Availability, fallback behavior, and key-type details are provided in the Supplementary Materials (Secure Enclave Integration).

6.6 Signing Key Hierarchy

The system uses two complementary signatures: (1) protocol signatures via the negotiated suite (Ed25519 for classic, ML-DSA-65 for PQC) to bind peer identity to the transcript, and (2) optional Secure Enclave P-256 signatures for device proof-of-possession. Protocol signatures are mandatory; Secure Enclave PoP is optional and provides a hardware-backed signal. Key hierarchy details and verification rules are provided in the Supplementary Materials (Signing Key Hierarchy).

6.7 Pre-Negotiation Signature Algorithm Selection and Two-Attempt Strategy

A fundamental challenge is that `sigA` must be generated before negotiation completes, yet the signature algorithm should match the negotiated suite. We resolve this with pre-negotiation signature selection and a two-attempt strategy. Each attempt is homogeneous (PQC-only or classic-only), and the signature algorithm is chosen based on the offered suites. The initiator first attempts PQC, then falls back to classic only for a whitelist of benign, locally generated capability errors after peer identity verification; timeout, signature verification, identity mismatch, replay, and other network-derived failures never trigger fallback. This policy is enforced by the handshake manager and logged as security events. Detailed error lists and type-level enforcement are provided in the Supplementary Materials (Two-Attempt Strategy). For strict policy deployments, we expose two operator-selectable modes: *Compliance Mode* (hard fail when PQC cannot be established) and *Bootstrap-Assisted Mode* (allow one temporary classic control channel only for paired KEM-key recovery, then require PQC rekey before business traffic).

Fallback Boundary Conditions (Explicit Scope):

- *We defend against:* (1) Attacker-induced timeout forcing downgrade (blocked by timeout blacklist); (2) Rapid downgrade cycling via repeated failures (5-minute per-peer cooldown); (3) Signature/identity attacks masquerading as capability failures (verification failures are blacklisted).
- *We do not defend against:* (1) Attacker persistently blocking PQC traffic for >5 minutes under default mode (availability vs. security trade-off; strict compliance mode provides full protection at cost of connectivity); (2) Legitimate PQC provider failures causing temporary classic fallback under default mode.
- *Cooldown rationale:* We set a conservative default of 5 minutes to bound downgrade cycling while limiting availability impact; the cooldown is a tunable policy knob. Operators requiring zero classic establishment can select strict Compliance Mode.

Attempt State and Identifiability:

- *Shared across attempts:* pinned identity fingerprint and pinned KEM public keys (trust record), and per-peer fallback cooldown state.
- *Not shared across attempts:* nonces, transcript hashes, handshakeId/replay tags, ephemeral key shares, and session keys; each attempt runs a fresh state machine.
- *Identifiability:* a passive observer can distinguish a classic fallback attempt from a PQC attempt by wire format. We do not transmit an explicit attempt counter; strict Compliance Mode removes classic attempts entirely when capability privacy outweighs compatibility.

Security Events:

Under default policy, classic fallback emits event “cryptoDowngrade” with reason, deviceId, and cooldown fields. Strict Bootstrap-Assisted recovery emits event “handshake_fallback” (bootstrap-assisted mode) with explicit trigger/result fields. This distinguishes strict compliance (fail-closed) traces from strict recovery traces with mandatory PQC rekey.

TABLE 3
Security Contract (Properties, Enforcement, Evidence).

Property	Enforced at	Evidence
G1 Negotiation integrity	Suite/keyshare validation + transcript-bound sigB	Fig. 1, HandshakeDriverTests
G2 Mutual authentication	Identity pinning + signature verification	Table 6 (wrong signature), Table 7
G3 Session key secrecy	KEM/DH + HKDF with transcript-bound salt	Property 1, Supplementary Materials (ML-DSA Key Sizes)
G4 Replay resistance	handshakeId cache + nonce binding	Property-Oriented Testing (Section 7)
G5 Strict-policy consistency	TwoAttempt policy gate + bootstrap control-only gate + mandatory rekey to PQC	Fig. 4, PolicyDowngradeBenchTests, BootstrapAssuranceTests
G6 Default safe fallback	Whitelist/blacklist + local capability gate + cooldown	Fig. 5, Table 6
G7 Legacy acceptance precondition	LegacyTrustPrecondition	Table 7
G8 Auditability	Security event emission + transcript-anchored AuditTrail hash-chain	Fig. 6, Table 8

TABLE 4
Default-policy fallback classes and remote-inducibility boundary.

Reason class	Default-policy outcome
Local capability/instantiation unavailability (pre-send; e.g., local suiteNegotiationFailed due to missing paired KEM keyshare)	ALLOW classic fallback + mandatory cryptoDowngrade event
Network-derived / after-send failures (timeout, remote negotiation mismatch, signature/identity/replay failures)	DENY fallback; emit failure event only

7 EVALUATION

7.1 Experimental Setup

We evaluate SkyBridge Compass along two primary tracks: **security-centric evidence** (fault injection, downgrade suppression, legacy precondition enforcement, and auditability) and **cost-centric evidence** (handshake latency, wire overhead, provider selection overhead, and data-plane throughput).

Environment: All experiments are run on Apple Silicon (M1/M3 class) with macOS 26.x, where CryptoKit PQC is available when the SDK exposes ML-KEM/ML-DSA types [16], [23]. Build configuration uses Release (-O) with non-essential logging disabled. Timing uses a monotonic clock (ContinuousClock) with 10 warmup runs; per-batch iterations are N=1000 for Classic/liboqs and N=5000 for CryptoKit (stability control). Reported percentiles (p50/p95/p99) are computed directly from samples.

Repeatability: We support multi-batch repeatability by restarting the benchmark test process for each batch. Supplementary Tables S12–S13 report the number of observed batches (B) and (when $B \geq 2$) mean and 95% confidence intervals across batches. To reproduce multi-batch CIs, re-run the harness with SKYBRIDGE_BENCH_BATCHES=3–5.

Reproducibility: We ship an opt-in benchmark suite under Tests/SkyBridgeCoreTests/. For a one-shot run, use Scripts/run_paper_eval.sh. The complete command reference and CSV artifact specifications are provided in the Supplementary Materials (Reproducibility).

iOS Reproducibility: We ship an on-device micro-benchmark in the iOS app (Settings → PQC → Self-test/Bench) that exports schema-v3 JSON artifacts. For the pinned run (ARTIFACT_DATE=2026-01-23), each suite uses warmup=10, N=1000, and three batches (B=3). We aggregate exported JSON with aggregate_ios_microbench.py into the date-

stamped iOS micro-benchmark CSV, then auto-generate the main table and Supplementary device metadata. In this frozen run (iOS/iPadOS 26.2.1), the iOS compatibility-matrix gate passed on both tested device classes (iPad + iPhone), with no schema or negotiation incompatibility observed in the benchmark pipeline.

Interpretation: Table 5 reports *primitive-level* iOS costs (encap/sign/verify/seal+open), not end-to-end handshake completion. It is therefore not interpreted as a direct substitute for macOS loopback handshake latency.

7.2 Claim Scope and External Validity

We separate *mechanism-level* from *deployment-level* claims. Mechanism-level claims cover negotiation integrity, policy-gated downgrade behavior, deterministic failure semantics, and audit-event completeness under the stated threat model. These claims are tied to the protocol contract and test/formal artifacts, not to Apple-specific APIs. Deployment-level claims cover absolute latency, wire overhead, and provider/runtime effects. They are evaluated only for the reported Apple-platform instance and should not be extrapolated to other OS stacks without remeasurement. To reduce portability ambiguity, we ship:

- a cross-platform contract-consistency JSON artifact (Supplementary Table S19);
- a dedicated external Linux direct-path run (Supplementary Table S7).

Accordingly, we use the real-network results as external-validity evidence rather than ecosystem-wide performance characterization: the current snapshot includes one STUN-coupled non-overlay direct path, overlay-mediated paths, and a dedicated external Linux direct-path run ($n=50$ per payload), and is used to validate directional robustness and failure semantics under deployment constraints.

7.3 Security Validation

We validate security properties through fault injection, property-based testing, and audit-signal fidelity analysis. This section presents the key results; detailed test matrices and methodology are provided in the Supplementary Materials (Regression Testing).

Failure-Mode Robustness. The actor-isolated handshake driver targets deterministic failure semantics; in our fault-injection tests we observed no unhandled errors, double-resume, or sensitive-material leaks. We inject faults including timeout, malformed framing, invalid signatures, out-of-order delivery, and concurrent cancellation ($n=1000$ per scenario per policy).

TABLE 5

On-device micro-benchmark on iOS 26+ (two devices, schema v3 artifacts). Each device runs warmup=10, N=1000, batches=3. Cells report mean/p99 in ms from pooled measured iterations. Data from `Artifacts/ios_microbench_2026-01-23.csv` (generated via `Scripts/aggregate_ios_microbench.py`).

Configuration	ipad16_3				iphone17_1			
	KEM encap	Sign	Verify	KEM-DEM	KEM encap	Sign	Verify	KEM-DEM
Classic (X25519 + Ed25519)	0.182 / 7.314	0.105 / 0.072	0.067 / 0.049	0.388 / 7.588	0.190 / 7.479	0.111 / 0.086	0.066 / 0.046	0.426 / 8.052
CryptoKit PQC (ML-KEM-768 + ML-DSA-65)	0.131 / 0.097	1.771 / 8.396	0.103 / 0.071	0.333 / 7.519	0.143 / 0.272	3.135 / 10.124	0.125 / 0.091	0.387 / 8.232
CryptoKit Hybrid (X-Wing + ML-DSA-65)	0.215 / 7.375	1.671 / 8.454	0.106 / 0.072	0.424 / 7.558	0.255 / 8.108	2.711 / 9.960	0.127 / 0.096	0.520 / 8.591

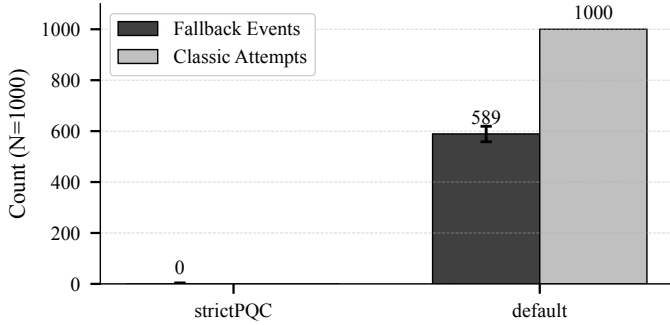


Fig. 4. Policy guard enforces strictPQC Compliance Mode: fallback events per 1000 runs, macOS 26.x, N=1000 per policy. Error bars show 95% Wilson binomial CI.

Unknown-suite handling: We parse unknown suite identifiers as an explicit `unknown(wireId)` value and reject them if they appear in `keyShares` or as `selectedSuite`. This prevents silent “unknown → classic” coercions that would otherwise weaken downgrade guarantees.

The delay-within-timeout and delay-exceed-timeout scenarios approximate RTT jitter and loss-induced stalls, while drop and duplicate cases approximate packet loss and reordering. The per-scenario event distributions are recorded in `Artifacts/fault_injection_2026-01-23.csv`.

Downgrade events are quantified separately in the policy bench (Fig. 4) to avoid conflating fallback behavior with transport corruption scenarios.

The plot in Fig. 4 uses the policy-downgrade dataset. File: `policy_downgrade_2026-01-23.csv`. It demonstrates that strictPQC Compliance Mode never emits fallback events even under forced PQC-unavailable errors; default policy shows non-zero downgrade events.

Fig. 5 encodes the explicit downgrade whitelist/blacklist: only PQC-unavailability and local pre-send suite-selection errors may fallback under default policy; strictPQC Compliance Mode denies classic fallback edges (Bootstrap-Assisted strict recovery is a separate control-only path with mandatory PQC rekey).

Fig. 6 combines fault-injection counts with the downgrade-acceptance event from the policy bench to visualize observability of failures and downgrades across policy modes.

Property-Oriented Testing. Correctness properties are validated with property-based tests using a reproducible seed (0xDEADBEEF):

- KEM-DEM round-trip correctness across all providers (100 trials)
- Signature verification integrity (100 payloads per provider)
- Secure zeroization on deallocation

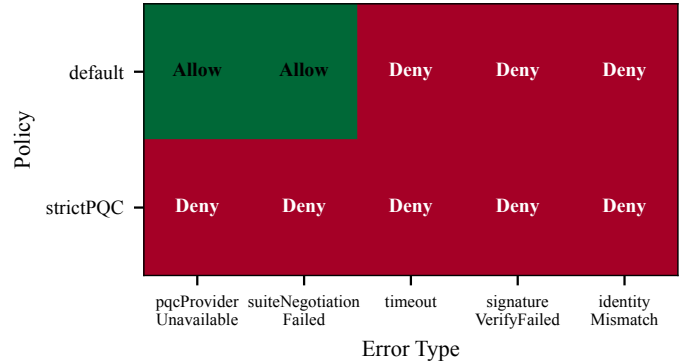


Fig. 5. Downgrade decision matrix (policy × error) with explicit allow/deny semantics, macOS 26.x. Each cell is labeled ALLOW or DENY for grayscale readability.

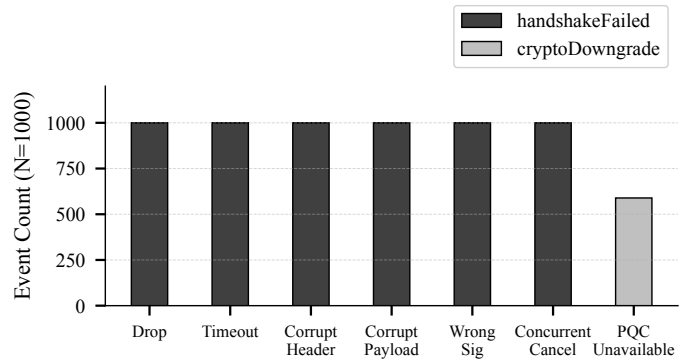


Fig. 6. Failure-mode histogram of security events, macOS 26.x, N=1000 per scenario.

- State-machine single-resume guarantee under concurrent stress (500 iterations)

A standalone Python decoder (`Scripts/wire_format_sanity.py`) provides non-Swift interoperability validation of wire formats.

ML-DSA Key Lifecycle. ML-DSA-65 key generation, storage, and sign/verify operations are validated against FIPS 204 [24]. Size-compliance results are reported in the Supplementary Materials (ML-DSA Key Sizes). All property tests (round-trip, tampering detection, key independence) achieve 100% pass rate. Keys are stored in Keychain with device-local accessibility and synchronization disabled (`kSecAttrSynchronizable=false`).

Legacy Fallback Security. Legacy P-256 signatures are only accepted when a security precondition is satisfied: (1) authenticated out-of-band channel (QR code, PAKE/PIN), or (2) existing TrustRecord with `legacyP256PublicKey`. Pure network stranger connections are rejected to prevent downgrade attacks.

TABLE 6

Failure-Mode Robustness and Observability. Runs = number of iterations per scenario and policy (n=1000). Table reports default policy; strictPQC (Compliance Mode) counts are recorded separately in `fault_injection_2026-01-23.csv`. All metrics are binary (1 = pass) except event counts. Data from `Artifacts/fault_injection_2026-01-23.csv` produced by `HandshakeFaultInjectionBenchTests`; semantic checks from `HandshakeDriverTests`.

Failure Mode	Runs	No Unexp. Error	No Double Resume	Zero Verified	$E_{\text{handshakeFailed}}$	$E_{\text{cryptoDowngrade}}$
Out-of-order	1000	1	1	1	0	0
Duplicate	1000	1	1	1	0	0
Drop	1000	1	1	1	1000	0
Delay within timeout	1000	1	1	1	0	0
Delay exceed timeout	1000	1	1	1	1000	0
Corrupt header	1000	1	1	1	1000	0
Corrupt payload	1000	1	1	1	1000	0
Wrong signature	1000	1	1	1	1000	0
Concurrent cancel	1000	1	1	1	1000	0
Concurrent timeout	1000	1	1	1	1000	0

TABLE 7

Legacy Fallback Precondition Test Results. All tests executed via `LegacyFallbackPreconditionTests`. Tests validate Property G7: Legacy Fallback Security Precondition.

Scenario	Precondition	Expected Result	Actual Result
Pure network stranger	None	Reject	OK Reject
QR code pairing (verified)	authenticatedChannel	Allow	OK Allow
PAKE pairing (verified)	authenticatedChannel	Allow	OK Allow
Existing TrustRecord with legacy key	existingTrustRecord	Allow	OK Allow
Existing TrustRecord without legacy key	None	Reject	OK Reject
Network discovery	None	Reject	OK Reject
Unverified authenticated channel	None	Reject	OK Reject

Property G7 (Legacy Fallback Security Precondition): A legacy P-256 signature SHALL only be accepted when a security precondition is satisfied. Pure network stranger connections SHALL be rejected. Validated with 12 test cases covering all precondition combinations (100% coverage of the precondition state space).

Audit-Signal Fidelity. Security events provide deterministic, audit-friendly (best-effort, non-cryptographic) telemetry of failures and policy decisions. Fig. 7 illustrates a representative timeout trace showing attacker action, policy gate response, and emitted event.

Evaluation approach: We evaluate audit-signal fidelity using a spec-to-event mapping consistency metric, *not* a statistical classifier. Each fault-injection scenario deterministically maps to exactly one expected trace template (event sequence), e.g., timeout $\rightarrow$ handshakeFailed=1, cryptoDowngrade=0. Match rate (MR)=1.00 means all N iterations matched the expected template; spurious rate (SR)=0.00 means no unexpected events were emitted. This is regression consistency against the protocol specification, not ML classification performance.

Table 8 summarizes results across all fault-injection scenarios: all scenario classes achieve 1.00 match and 0.00 spurious rates, confirming correct event semantics.

Due to space constraints, the complete regression test matrix, TLV encoding validation, and implementation component details

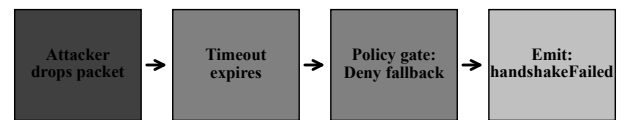


Fig. 7. Audit-signal event trace (representative timeout case) showing attacker action, policy gate, and emitted event.

are provided in the Supplementary Materials (Regression Testing and Artifact Map).

7.4 Resource Amplification and Throttling

We separate adversarial cost vectors into (i) pre-authentication parser cost and (ii) post-authentication retry/cooldown pressure. Pre-auth cost is bounded by strict field-length checks, overflow-safe parsing, and bounded key-share cardinality before cryptographic operations are invoked. Post-auth pressure is bounded by per-peer fallback cooldown and rate-limited security event emission.

TABLE 8

Audit-Signal Fidelity Summary. Match/spurious rates reflect *deterministic* protocol event mapping (each scenario maps to exactly one expected trace template), not statistical classification. Perfect scores indicate regression consistency against the protocol specification.

Scenario Class	Expected Signal	N	Match (MR)	Spurious (SR)
Drop/Timeout (default)	handshakeFailed=1, cryptoDowngrade=0	3000	1.00	0.00
Drop/Timeout (strictPQC)	handshakeFailed=1, cryptoDowngrade=0	3000	1.00	0.00
Corrupt/Wrong Sig	handshakeFailed=1, cryptoDowngrade=0	6000	1.00	0.00
Ordering Benign	handshakeFailed=0, cryptoDowngrade=0	6000	1.00	0.00
PQC unavailable (default)	cryptoDowngrade=1	1000	1.00	0.00
PQC unavailable (strictPQC)	cryptoDowngrade=0	1000	1.00	0.00

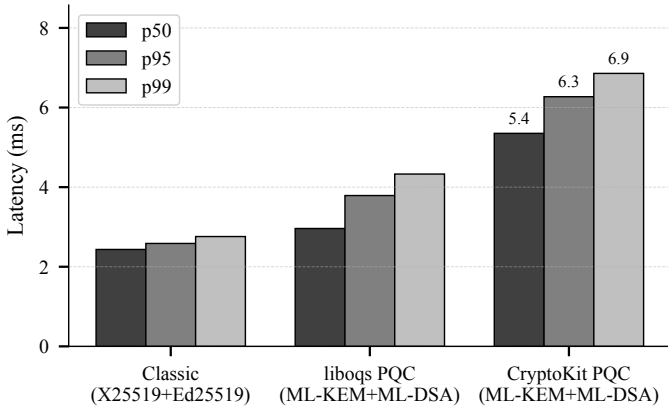


Fig. 8. Handshake latency percentiles for Classic, liboqs PQC, and CryptoKit PQC (warmup=10; N=1000 for Classic/liboqs, N=5000 for CryptoKit). Data source: `handshake_bench_2026-01-23.csv`. Values align with Table 9 and Supplementary Table S1.

These controls are intended to bound resource amplification from malformed inputs and retry storms without introducing silent downgrade paths. We do not claim Internet-scale saturation resistance from this revision alone; adversarial burst-throughput characterization remains future work.

7.5 Performance Benchmarks

Table 9 consolidates all performance metrics. We measure end-to-end handshake completion time using an in-memory loopback transport to isolate cryptographic and state-machine overhead. Each configuration reports distributional statistics after 10 warmup runs (Classic/liboqs: N=1000; CryptoKit: N=5000).

v1/v2 Bridge Overhead. To isolate the protocol-v2 cost, we run a controlled liboqs v1/v2 bridge comparison with identical hardware/runtime settings (N=1000, warmup=10). Supplementary Table S10 reports mean/p95 latency, RTT, and payload-handshake bytes. Consistent with the wire-format analysis in Section 5.3, v2 adds a fixed +75 B payload-handshake overhead (MessageA +43 B, MessageB +32 B), while latency impact remains in the low-millisecond range.

Latency Distribution. Fig. 8 shows latency percentiles across configurations. Classic handshakes remain near 2.454 ms (2.780 ms at p99). CryptoKit PQC remains around 6.857 ms at p99, with variance dominated by ML-DSA signature verification overhead. Extended statistics (mean/p50/p95/p99, std) are available in Supplementary Tables S1–S2 (reported in the supplementary PDF).

Wire-Format Breakdown. The handshake payload size increases from Classic (687 B) to PQC (12002 B), dominated by

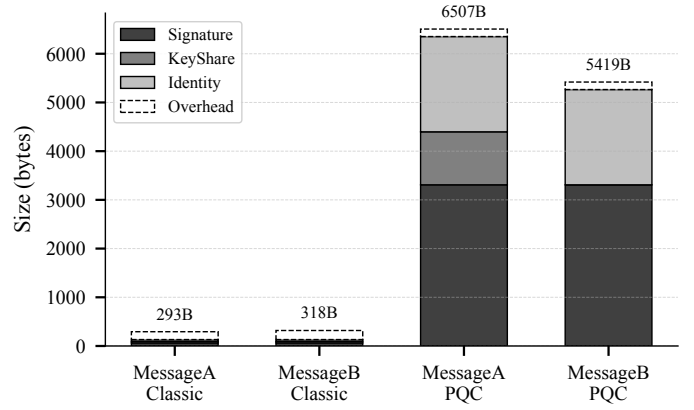


Fig. 9. Handshake message size breakdown (signature, keyshare, identity fields, framing overhead) for MessageA/MessageB across Classic and PQC configurations. Counts reflect MessageA/B payloads only. The *payload-only* handshake size adds two 38 B Finished frames (accounted for in Table 9); detailed field sizes are reported in Supplementary Table S3. Data source: `message_sizes_2026-01-23.csv`.

ML-DSA signatures (3309 B) and ML-KEM ciphertexts (1088 B). Fig. 9 shows the per-field breakdown. **Path-MTU implication.** The payload-only PQC handshake (12002 B) spans multiple transport segments under common MTUs. In our design, handshake messages run over stream transports (TCP/QUIC stream), so segmentation/reassembly is delegated to transport rather than custom handshake-layer fragmentation; the practical risk is latency inflation on lossy paths, which is why we report controlled impairment, kernel-level, and real-network evidence in Section 7.7.

Note: In the 2026-01-23 snapshot, X-Wing hybrid KEM adds 128 B to MessageA’s keyshare (1216 B vs 1088 B for ML-KEM-768). Measured handshake sizes are reported in Supplementary Table S3 (MessageA.XWing/MessageB.XWing).

Data-Plane and Provider Selection. Post-handshake symmetric AEAD (AES-256-GCM) throughput reaches 3.7 GB/s for 1 MiB payloads on Apple Silicon, with CPU cost of 0.25 ns/byte. In v1 the data-plane AEAD is fixed to AES-256-GCM and recorded separately from the handshake suite; thus throughput is independent of the negotiated KEM/signature suite. Provider selection overhead is sub-microsecond for cached paths (0.17 μ s p50) and under 6 μ s even for fallback scenarios. Detailed breakdowns are provided in the artifact CSV files.

7.6 System-level Impact and Amortization

The preceding microbenchmarks isolate handshake and cryptographic costs. To align evaluation with the *remote desktop / file transfer* framing, we additionally measure user-visible, session-

TABLE 9

Performance Summary. All benchmarks on Apple Silicon (M1/M3), macOS 26.x, configuration-specific N after 10 warmup runs (Classic=1000, liboqs PQC=1000, CryptoKit PQC=5000). Wire Size counts payload-only handshake bytes (MessageA + MessageB + 2×Finished); loopback wire sizes including transport overhead are reported separately in Table 1 and Supplementary Table S11. Data-plane AEAD is fixed to AES-256-GCM in v1, so throughput is independent of the negotiated handshake suite; throughput measured post-handshake on 1 MiB payloads.

Configuration	Handshake Latency		RTT		Wire Size (bytes)	Throughput (GB/s)
	mean (ms)	p95 (ms)	p50 (ms)	p95 (ms)		
Classic	2.45	2.59	0.55	0.59	687	3.7
liboqs PQC	2.98	3.66	0.91	1.47	12,002	3.7
CryptoKit PQC	5.44	6.27	2.40	3.09	12,009	3.7

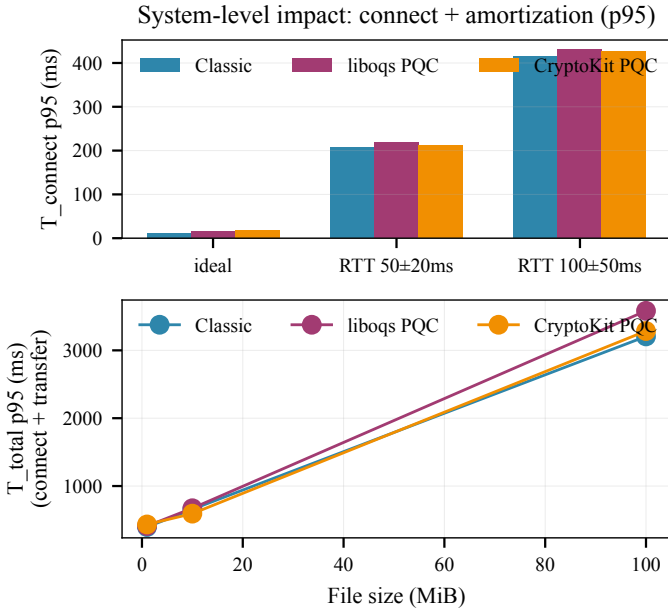


Fig. 10. System-level impact and amortization (p95). Top: $T_{connect}$ across RTT+jitter conditions. Bottom: total session time ($T_{total} = connect + transfer$) versus file size under RTT 50±20 ms, showing that one-time handshake overhead is amortized by larger transfers. Data from Artifacts/system_impact_2026-01-23.csv.

level metrics and quantify how one-time handshake cost amortizes over longer sessions and larger transfers.

Metrics: (i) $T_{connect}$ (connect start → handshake established, including explicit Finished key confirmation and event emission), (ii) T_{first_frame} (connect start → first decrypted remote-desktop frame processed by the rendering path), and (iii) T_{total} (connect start → completion of a bulk transfer over the negotiated session keys).

Method: We reuse the production HandshakeDriver and negotiated SessionKeys, with the following setup:

- Workload: one encrypted “first frame” plus bulk transfer workloads (1–100 MiB).
- Channel model: deterministic RTT+jitter on a reliable stream.
- Throughput control: bulk payloads are rate-limited (MiB/s) to avoid per-chunk RTT artifacts.
- Output: date-stamped system-impact CSV artifact generated by SystemImpactBenchTests.

Traffic Analysis Mitigation (SBP2 Padding). To reduce attempt identifiability on the wire, SkyBridge optionally applies a constant-size framing layer (SBP2) that buckets framed

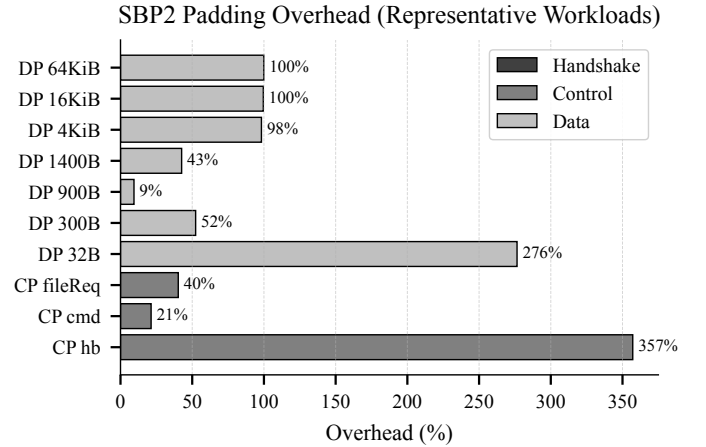


Fig. 11. SBP2 padding overhead across representative handshake, control, and data-plane workloads (macOS 26.x). Overhead is computed as $(\text{paddedBytes}/\text{rawBytes} - 1) \times 100\%$ aggregated per label over a deterministic trace (N=1000 handshake iterations; data-plane workload uses encrypted control messages and large-frame mixes). Generated from Artifacts/traffic_padding_2026-01-23.csv.

payloads into powers-of-two-sized envelopes. Here, **SBP1** denotes *handshake-frame padding* (MessageA/MessageB/Finished are padded to fixed buckets before transport), while **SBP2** denotes *traffic framing and padding* applied to all framed payloads (control and encrypted data-plane) with a per-frame header and bucketed padding up to a configured cap. We measure SBP2 behavior on a real protocol trace: an end-to-end handshake (SBP1 + SBP2 enabled) on an in-memory loopback transport, followed by bidirectional AES-GCM-encrypted data-plane frames derived from the negotiated session keys. Supplementary Table S4 reports the resulting quantization behavior and overhead; the full per-label bucket histograms are included in Artifacts/traffic_padding_2026-01-23.csv.

Sensitivity Study (Bucket Cap). We vary the maximum bucket size for SBP2 ($cap = 64\text{ KiB}, 128\text{ KiB}, 256\text{ KiB}$) and re-run the same deterministic protocol trace. We report two overhead definitions: *all-frames overhead* (includes passthrough frames) and *padded-only overhead* (restricted to frames that entered bucket padding), alongside *over-cap rate* (fraction of frames whose framed payload exceeds cap) and *bucket entropy* (Shannon entropy of padded bucket sizes per label). When over-cap is high, all-frames overhead may approach zero while padded-only overhead remains non-zero; this is a cap tradeoff, not a measurement bug. Supplementary Table S5 summarizes this overhead–privacy tradeoff and distinguishes remote-desktop-like vs file-chunk-like large-frame distributions.

TABLE 10

System-level impact and amortization (p50/p95, ms). $T_{connect}$ is measured from connect start to handshake established (including Finished key confirmation and event emission). T_{total} is measured from connect start to completion of an encrypted bulk transfer. Network conditions emulate deterministic RTT+jitter on a reliable stream; large data-plane payloads are bandwidth-limited (MiB/s) to avoid per-chunk RTT artifacts. Runs: $N_{connect}=1000$; N_{total} (RTT 50±20 ms) = 1MiB:N=1000, 10MiB:N=500, 100MiB:N=100. Data from `Artifacts/system_impact_2026-01-23.csv`.

Suite	$T_{connect}$ (p50/p95)			T_{total} (p50/p95), RTT 50±20 ms		
	ideal	RTT 50±20 ms	RTT 100±50 ms	1 MiB	10 MiB	100 MiB
Classic	11.9/12.2	191.9/198.6	407.8/414.6	378.3/395.6	634.8/657.3	3179/3206
liboqs PQC	14.0/15.1	195.1/202.1	415.4/430.9	372/410	638.4/672.6	3192/3583
CryptoKit PQC	16.3/18.4	196.2/205.8	412.1/426.8	371/434	581.0/593.2	3208/3285

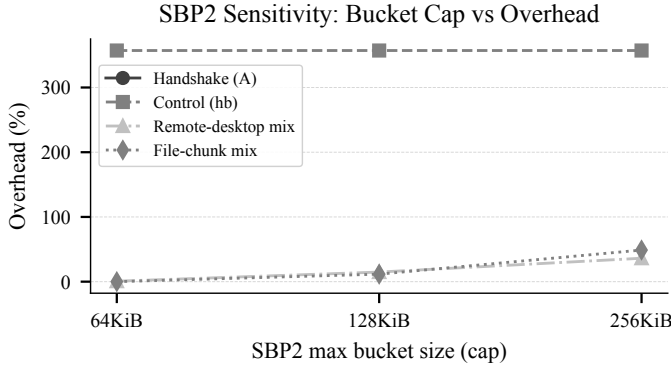


Fig. 12. SBP2 bucket-cap sensitivity on macOS 26.x. Lower over-cap rate means broader padding coverage across 64 KiB, 128 KiB, and 256 KiB caps for remote-desktop and file-transfer mixes.

TABLE 11
Handshake vs data-plane AEAD selection (v1).

Layer	AEAD	Negotiation source
Handshake payloads	AES-256-GCM	Suite tuple
Data-plane traffic	AES-256-GCM	Fixed (v1)

Operational decision guide (rule-of-thumb). For interactive remote-desktop-first workloads, prefer SBP2 disabled or 64 KiB cap. For mixed control + moderate transfer workloads, 128 KiB is a balanced default. For bulk-transfer-heavy, privacy-prioritized workloads, 256 KiB improves padding coverage at the cost of higher bandwidth overhead.

7.7 Network Impairments: Controlled, Kernel-level, and Real-network Evidence

To reduce external-validity ambiguity, we report a three-layer evidence stack with one shared profile source (`Scripts/netem_profiles.json`): (i) controlled in-process simulation, (ii) kernel-level emulation cross-check (`dummynet/netem`), and (iii) real-network micro-study.

Controlled simulation (in-process). We simulate four network conditions with a retry-aware model (max 2 retries) to isolate protocol logic under reproducible loss/jitter/reorder perturbations. Loss is Bernoulli per packet; jitter is sampled uniformly from $\pm J$ ms around base RTT; reordering is modeled as added random delay (50–150 ms) on 10% of packets.

Kernel-level cross-check.

- Tools: `dnetl/pfctl` (macOS `dummynet`) and `tc netem` (Linux), with aligned profile IDs.

- Orchestration: kernel emulation script; outputs are summarized in Supplementary Table S8.
- Capability note: reorder is reported only for `netem`; `dummynet` reorder cells are *n/a*.
- Interpretation: on some macOS setups, `dummynet` with localhost endpoints may under-shape absolute delay; therefore, delay-scale reading is anchored by `netem` and controlled simulation, while cross-tool completion robustness remains the primary check.

Real-network micro-study.

- Metrics: STUN path telemetry and TCP end-to-end timing for payload-only handshake baselines (classic 687 B, PQC 12,002 B), reported in Supplementary Table S6.
- Artifact source: `Artifacts/realnet_*`.
- Direct label in this snapshot: `wifi_direct` (remote EC2 over TCP 443).
- Overlay labels in this snapshot: `ec2_ssh_tunnel` and `hotspot_ssh_tunnel`.
- Reporting rule: direct and overlay paths are reported separately to distinguish external direct-path evidence from tunnel-mediated behavior under restrictive inbound NAT/firewall conditions.
- Accounting: the harness sends 4B `length-prefix` + `payload`; payload-byte labels exclude the fixed prefix.
- External check: dedicated direct-path EC2 Linux run (no tunnel, 54.92.79.99:44444, $n=50$ per payload) in Supplementary Table S7.

8 LIMITATIONS AND FUTURE WORK

8.1 Current Limitations

- iOS PQC Availability (iOS < 26):** We do not currently bundle `liboqs` for iOS; therefore PQC suites are only available on iOS 26+ devices via `CryptoKit PQC` [16]. To support reproducible mobile evaluation, we ship an *on-device* self-test and micro-benchmark harness in the iOS app (Settings → PQC → “Self-test/Bench”), which exports JSON artifacts for inclusion in Supplementary tables.
- X-Wing Hybrid KEM:** We have reserved wire-format identifiers for X-Wing hybrid KEM (wireId 0x0001, combining X25519 + ML-KEM-768). `CryptoKit` on Apple 26+ exposes HPKE cipher suites including X-Wing (ML-KEM-768 with X25519), motivating a standards-informed hybrid path. Our current implementation uses a domain-separated hybrid KEM composition (X25519

TABLE 12
Controlled in-process network simulation results (N=10,000 per condition, max retries=2). Completion and latency ranges span classic/liboqs/CryptoKit suites.

Condition (loss, RTT, jitter/reorder)	Completion rate (min)	p95 latency (ms, range)	handshakeFailed events / 10k (max)
mild (1%, 50 ms, ± 20 ms)	1.0000	251–253	0
moderate (3%, 100 ms, ± 50 ms)	1.0000	529–532	0
severe (5%, 200 ms, ± 100 ms)	0.9989	1031–1033	11
reorder (10% reorder, 50 ms, ± 20 ms)	1.0000	368–370	0

+ ML-KEM-768) under wireId 0x0001 and reports on-device measurements in Table 5; full integration with CryptoKit’s PQ-HPKE ciphersuite API remains future work.

- 3) **Key Rotation:** Session key rotation during long-lived connections is not addressed in the current design.
- 4) **Forward Secrecy coverage is suite-dependent:** v2 introduces a hybrid composition (static KEM secret + ephemeral contribution) to strengthen post-compromise secrecy versus v1, but legacy v1 deployments remain static-recipient KEM and do not provide equivalent FS semantics.
- 5) **Cross-Platform Interoperability and Generalization Boundaries:** The migration contract and wire format are designed to be portable, and this revision includes contract-level interop evidence across iOS/macOS, Android, and Ubuntu (suite-catalog parity, wire endianness, signature-selection contract, timeout-blocked fallback, cooldown guard, and trust-pinning path; Supplementary Table S19). However, absolute latency/throughput numbers in the main performance tables remain Apple-centric and should not be transferred to non-Apple deployments without remeasurement.
- 6) **Network Condition Evaluation:** We provide simulated impairment results, kernel-level cross-check, and a real-network micro-study in Section 7.7; however, the real-network sample size remains limited (one non-overlay direct label plus overlay-mediated SSH-tunnel labels in the STUN-coupled micro-study, plus one dedicated direct-path EC2 Linux run with $n=50$ per payload) and does not yet cover large-scale mobility, long-lived paths, or broad ISP diversity. In addition, while dummynet and netem both preserve high completion under shared profile IDs, the observed delay-profile separation is tighter for our current dummynet setup than for netem, so delay-scale claims are anchored primarily by controlled simulation and netem.

8.2 Future Directions

- 1) **Cross-Platform Performance Expansion:** Extend quantitative latency/throughput evaluation on Android/Ubuntu production devices (beyond current contract-level interop evidence) and report cross-device variance under identical workload definitions.
- 2) **PFS Hardening Beyond v2:** Extend v2 toward full ephemeral-ephemeral designs and quantify compromise windows under delayed key-exposure models.
- 3) **Cross-Peer Audit Attestation:** Build on the current transcript-anchored local hash-chain by exchanging and

cross-signing compact audit digests between peers for stronger remote verifiability.

- 4) **Traffic Analysis Mitigations:** Reduce two-attempt identifiability via optional padding or constant-size framing, while preserving strict Compliance-Mode semantics for deployments that require zero classic establishment.
- 5) **Real-Network and Kernel-Emulation Expansion:** Extend the current micro-study and kernel cross-check to larger cohorts, more ISP topologies, and longer sessions; compare TCP vs QUIC control channels under matched profile IDs and evaluate stability over multi-day runs. Immediate targets are to add at least one additional non-overlay real-network labels and to calibrate dummynet profile separation with non-localhost endpoints plus independent packet-capture validation.
- 6) **Formal Verification:** Apply model checking to the handshake state machine to prove absence of deadlocks and race conditions.
- 7) **Performance Optimization:** Profile ML-KEM-768 encapsulation latency and explore hardware acceleration opportunities.
- 8) **Certificate-Based Identity:** Integrate with device certificates for enterprise deployment scenarios.

9 CONCLUSION

SkyBridge Compass shows that cryptographic agility and post-quantum readiness are attainable in P2P systems without sacrificing API stability or operational transparency. The implementation is production-focused. The layered CryptoProvider architecture enables adoption of new primitives as platforms evolve. The actor-isolated handshake state machine reduces concurrency hazards and limits sensitive-material exposure.

We do not introduce new cryptographic primitives; instead, we contribute an enforceable migration protocol/strategy layer with formalized security definitions, machine-checkable proofs, and a v2 forward-secrecy-oriented composition path that remains bridge-compatible with v1 deployments.

The primary contribution is this migration contract and its verification workflow, not a platform-specific API binding. Our Apple-platform deployment indicates that native PQC APIs can be integrated with modest overhead when available, with graceful fallback to library-based or classic implementations on older systems. These measurements are deployment-instance results and should be re-established when porting to other ecosystems. The structured security event model is designed to make cryptographic decisions auditable, supporting both real-time monitoring and post-incident analysis.

SkyBridge Compass prioritizes deterministic semantics and auditability in cryptographic migration. The default policy favors availability and explicit, observable downgrade signals, at the

cost of attempt identifiability on the wire. Strict policy is now explicit in two operator-facing forms: Compliance Mode (fail closed, no classic establishment) and Bootstrap-Assisted Mode (recover paired KEM state via control-only bootstrap, then rekey to PQC before business traffic).

As post-quantum cryptography transitions from standardization to deployment, systems like SkyBridge Compass provide a practical template for managing this transition while maintaining security and usability across heterogeneous device fleets.

The limitations noted in Section 8 do not change the core contribution: an auditable migration contract with concurrency-safe handshake semantics that are reproducible from released artifacts under our stated threat model and measurement setup.

ACKNOWLEDGMENT

This work received no external funding.

DATA AND ARTIFACT AVAILABILITY

Artifact release:

- URL: <https://github.com/billlza/Skybridge-Compass>
- Git ref: 817e11aa5806
- Commit: 817e11aa5806
- Source archive: 817e11aa5806.zip
SHA256=05d6961a26ec0f5b57b8821cfc8337de
c1ba37ec8deb5f9dd5e7b1c7bdcdb2378
- Source archive: 817e11aa5806.tar.gz
SHA256=5b9130d64e5217376da98e20286f078f
d8693d998796d64d4e503e981f2dea2b

See the repository README for environment details and CSV schemas.

CONFLICT OF INTEREST

The author declares no competing interests.

REFERENCES

- [1] Bluetooth SIG, “Bluetooth Core Specification v5.4,” 2023.
- [2] Apple Inc., “Continuity,” Apple Platform Security Guide, 2024.
- [3] T. Taubert and C. A. Wood, “SPAKE2+, an Augmented PAKE,” RFC 9383, IETF, Sept. 2023. [Online]. Available: <https://datatracker.ietf.org/doc/rfc9383/>
- [4] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, IETF, 2018.
- [5] J. Iyengar and M. Thomson, “QUIC: A UDP-Based Multiplexed and Secure Transport,” RFC 9000, IETF, 2021. [Online]. Available: <https://datatracker.ietf.org/doc/rfc9000/>
- [6] E. Barker and A. Roginsky, “Transitioning the Use of Cryptographic Algorithms and Key Lengths,” NIST SP 800-131A Rev. 2, 2019.
- [7] E. Barker et al., “Considerations for Achieving Cryptographic Agility: Strategies and Practices,” NIST Cybersecurity White Paper (CSWP) 39, Dec. 2025, doi: 10.6028/NIST.CSWP.39.
- [8] T. Perrin, “The Noise Protocol Framework,” 2018. [Online]. Available: <https://noiseprotocol.org/noise.pdf>
- [9] NIST, “Post-Quantum Cryptography Standardization,” 2024. [Online]. Available: <https://csrc.nist.gov/projects/post-quantum-cryptography>. Accessed: Dec. 2025.
- [10] Signal Foundation, “PQXDH Key Agreement Protocol,” Signal Technical Documentation, 2023.
- [11] Cloudflare, “Post-Quantum Cryptography Goes GA,” Sept. 2023. [Online]. Available: Cloudflare blog post. Accessed: Dec. 2025.
- [12] P. Schwabe, D. Stebila, and T. Wiggers, “Post-Quantum TLS Without Handshake Signatures,” in Proc. ACM CCS 2020, pp. 1461–1480.
- [13] E. Alkim, L. Ducas, T. Pöppelmann, and P. Schwabe, “Post-Quantum Key Exchange—A New Hope,” in Proc. USENIX Security 2016, pp. 327–343.
- [14] J. Bos et al., “CRYSTALS—Kyber: A CCA-Secure Module-Lattice-Based KEM,” in Proc. IEEE EuroS&P 2018, pp. 353–367.
- [15] L. Ducas et al., “CRYSTALS—Dilithium: A Lattice-Based Digital Signature Scheme,” IACR Trans. CHES, vol. 2018, no. 1, pp. 238–268.
- [16] Apple Inc., “Quantum-secure cryptography in Apple operating systems,” Apple Platform Security, Jan. 2026. [Online]. Available: Apple Platform Security (secc7c82e533). Accessed: Feb. 2026.
- [17] D. Beyer et al., “Software Model Checking,” in Handbook of Model Checking, Springer, 2018.
- [18] B. Beurdouche et al., “A Messy State of the Union: Taming the Composite State Machines of TLS,” in Proc. IEEE S&P 2015, pp. 535–552.
- [19] Apple Inc., “Swift Concurrency,” The Swift Programming Language, 2024.
- [20] E. Rescorla and N. Modadugu, “Datagram Transport Layer Security Version 1.2,” RFC 6347, IETF, Jan. 2012.
- [21] M. Green and M. Smith, “Developers Are Not the Enemy!: The Need for Usable Security APIs,” IEEE Security & Privacy, vol. 14, no. 5, pp. 40–46, Sept./Oct. 2016.
- [22] K. Bhargavan, C. Fournet, M. Kohlweiss, A. Pironti, and P.-Y. Strub, “Implementing TLS with Verified Cryptographic Security,” in Proc. IEEE S&P 2013, pp. 445–459.
- [23] Apple Inc., “CryptoKit Framework,” Apple Developer Documentation, 2025. [Online]. Available: <https://developer.apple.com/documentation/cryptokit>. Accessed: Dec. 2025.
- [24] NIST, “Module-Lattice-Based Digital Signature Standard,” FIPS 204, Aug. 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/204/final>
- [25] D. McGrew, “Achieving Crypto Agility,” in Proc. RSA Conference, 2019.
- [26] R. Barnes, K. Bhargavan, B. Lipp, and C. Wood, “Hybrid Public Key Encryption,” RFC 9180, IETF, Feb. 2022. [Online]. Available: <https://datatracker.ietf.org/doc/rfc9180/>
- [27] Apple Inc., “Get ahead with quantum-secure cryptography,” WWDC 2025 Session 314, June 2025. [Online]. Available: <https://developer.apple.com/videos/play/wwdc2025/314/>. Accessed: Dec. 2025.
- [28] NIST, “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” FIPS 203, Aug. 2024. [Online]. Available: <https://csrc.nist.gov/pubs/fips/203/final>
- [29] NIST, “Recommendations for Key-Encapsulation Mechanisms,” SP 800-227, Final, Nov. 2025. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/227/final>
- [30] K. Cremers, M. Horvat, J. Hoyland, and S. Scott, “A Comprehensive Symbolic Analysis of TLS 1.3,” IACR ePrint 2020/1044, 2020. [Online]. Available: <https://eprint.iacr.org/2020/1044>
- [31] P. Schwabe, D. Stebila, and T. Wiggers, “KEMTLS: Authenticated Key Exchange without Signatures in TLS 1.3,” 2020. [Online]. Available: <https://kemtls.org/publication/kemtls/>
- [32] S. Meier, B. Schmidt, C. Cremers, and D. Basin, “The Tamarin Prover for the Symbolic Analysis of Security Protocols,” in Proc. CAV 2013, LNCS 8044, pp. 696–701.
- [33] M. Barbosa, D. Connolly, J. Duarte, A. Kaiser, P. Schwabe, K. Varber, and B. Westerbaan, “X-Wing: The Hybrid KEM You’ve Been Looking For,” IETF Internet-Draft, draft-connolly-cfrg-xwing-kem-09, Jan. 2026. [Online]. Available: <https://datatracker.ietf.org/doc/draft-connolly-cfrg-xwing-kem/>. Accessed: Feb. 2026.
- [34] D. J. Bernstein, “Curve25519: New Diffie-Hellman Speed Records,” in Proc. PKC 2006, LNCS 3958, Springer, 2006, pp. 207–228.
- [35] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, “High-Speed High-Security Signatures,” J. Cryptographic Engineering, vol. 2, no. 2, pp. 77–89, 2012.
- [36] H. Krawczyk, “SIGMA: The ‘SIGn-and-MAC’ Approach to Authenticated Diffie-Hellman and Its Use in the IKE Protocols,” in Proc. CRYPTO 2003, LNCS 2729, pp. 400–425.
- [37] H. Krawczyk and P. Eronen, “HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” RFC 5869, IETF, May 2010.
- [38] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” NIST SP 800-38D, Nov. 2007.
- [39] N. Aviram et al., “DROWN: Breaking TLS Using SSLv2,” in Proc. USENIX Security 2016, pp. 689–706.
- [40] D. Adrian et al., “Imperfect Forward Secrecy: How Diffie-Hellman Fails in Practice,” in Proc. ACM CCS 2015, pp. 5–17.
- [41] C. Meyer and J. Schwenk, “Lessons Learned From Previous SSL/TLS Attacks—A Brief Chronology of Attacks and Weaknesses,” IACR Cryptology ePrint Archive, Report 2013/049, 2013.
- [42] T. Tarmann, R. Hutchinson, L. Pierson, P. Sholander, and E. Witzke, “Algorithm-Agile Encryption in ATM Networks,” IEEE Computer, vol. 31, no. 9, pp. 57–64, Sept. 1998.
- [43] B. Sullivan and V. Liu, “Web Application Security: A Beginner’s Guide,” McGraw-Hill, 2011, ch. 6 (Cryptographic Agility).